GenAI, Agents & the Maturity Model

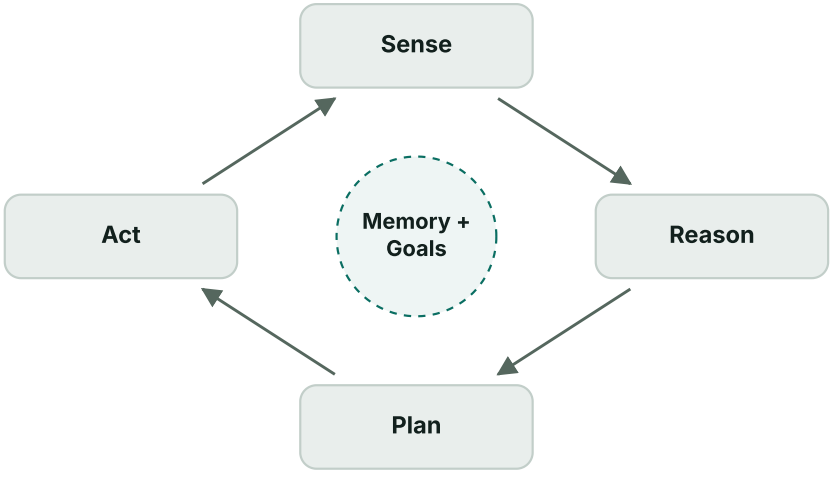
1. Three things, not one

- **LLM call**: a single request and response. Powerful but stateless and passive: exactly one turn
- **Automated workflow**: a predefined sequence where **your code decides the path** and LLM calls fill specific slots (classify, extract, draft). Predictable and auditable
- **AI agent**: an LLM placed in a loop with tools, where **the model decides** what to do next to reach a goal
- If the path is knowable in advance, a workflow is cheaper, faster and easier to debug. Reserve agents for genuinely open-ended tasks

2. Agent anatomy

- **Goals**: what it's trying to achieve
- **Sense**: perception, gathering context from APIs, databases, sensors ('context is king')
- **Reason**: the LLM cognition core · **Plan**: decide the next steps · **Act**: execute through tools
- **Memory**: state and past experience · **Coordinate**: only in multi-agent systems

✎ DIAGRAM

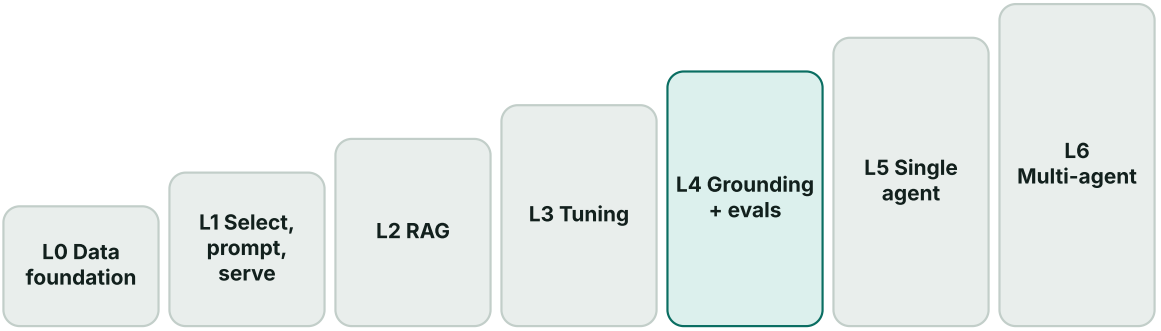


The agent loop: the result of each action becomes new context, grounded in memory and goals at every step.

3. The GenAI Maturity Model (L0 to L6)

It's not just a description of where you are; it's a **build order**, because each level is reached by implementing specific patterns.

✎ DIAGRAM



Level 4 is where outputs become trustworthy; agents (L5 to L6) need AgentOps on top.

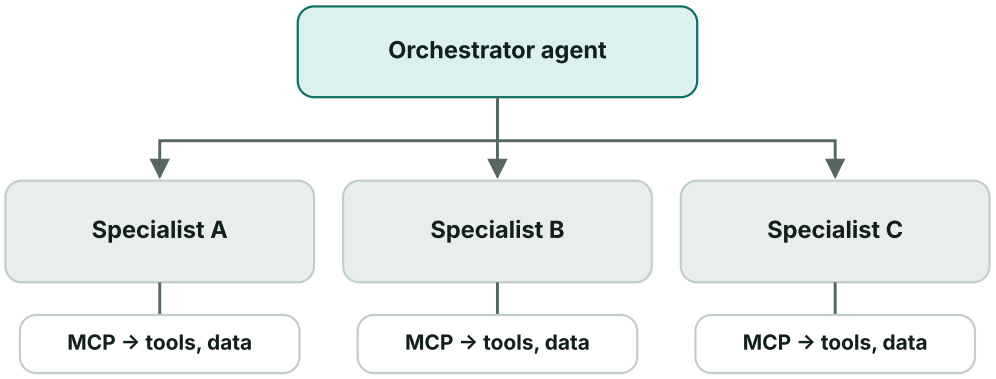
Each level is unlocked by specific patterns.

- **Pattern-first architecture**: pick a target capability ('fully auditable and safe for financial transactions'), reverse-engineer the four or five patterns that deliver it, and build exactly those

4. The agentic stack

- **Function calling** (the agent acts): you give the model tools with names, descriptions and parameter schemas; it emits a structured JSON call; your code runs it and returns the result. The description is what it routes on
- **MCP** (Model Context Protocol, agent → tools, vertical): a standard for describing, discovering and invoking tools and data sources
- **A2A** (Agent-to-Agent, agent → agents, horizontal): a shared language for delegating tasks between agents across frameworks and vendors. Discovery via an **agent card** (advertises capabilities) on an **agent server** (the endpoint)

✎ DIAGRAM



A2A connects the orchestrator to specialists (horizontal); MCP connects each agent to its tools (vertical).

5. Why production is hard

- Low-latency inter-agent communication, diverse data, reliable tool use, hallucination, **observability** into why a multi-agent system did something, an expanded attack surface, and the **AgentOps** discipline to run it all
- You don't need every pattern, only the ones that close the gap between your demo and the reliability your use case demands

SAY IT

"MCP connects an agent to its tools; A2A connects agents to each other. I'd only build an agent where a workflow can't do the job."